String Problem and Solutions

1. Reverse a String

Approach 1: Two Pointer Approach

- Trick: Use two pointers: one starting from the beginning and the other from the end. Swap characters until the pointers meet.
- Time Complexity: O(n)
- Space Complexity: O(1)

Pseudocode (C++)

```
void reverseString(string &s) {
int left = 0, right = s.size() - 1;
while (left < right) {
swap(s[left], s[right]);
left++;
right--;
    }
}
```

Dry Run

Input: "hello"

Steps:

Swap h and o: "oellh"

Swap e and l: "olleh"

Output:

"olleh"

2. Check whether a String is Palindrome or not

- Approach 1: Two Pointer Comparison

- Trick: Compare characters from the beginning and the end using two pointers.

- Time Complexity: O(n)

- Space Complexity: O(1)

Pseudocode (C++)

```
bool isPalindrome(string s) {
int left = 0, right = s.size() - 1;
while (left < right) {
if (s[left] =! s[right]) return false;
left++;
right--;
    }
return true;
}
```

Dry Run

Input: "madam"

Steps:

- Compare m and m:

Match

- Compare a and a: Match
- Compare d with itself: Match

Output: true

3. Find Duplicate Characters in a String

- Approach 1: Frequency Array
- Trick: Use a frequency array to count occurrences of characters.
- Time Complexity: O(n)
- Space Complexity: O(1) (constant for 26 letters or ASCII 256)

Pseudocode (C++):

```
void findDuplicates(string s) {
vector freq(256, 0);
for (char c   s) freq[c]++;
for (int i = 0; i < 256; i++) {
if (freq[i] > 1) cout << (char)i << " appears " << freq[i] << " times\n";
    }
}
```

Dry Run

Input: "programming"

Frequency:

r: 2, g: 2, m:

2

Output: r, g, m

4. Balanced Parenthesis Problem

- Approach 1: Stack
- Trick: Push opening brackets to the stack. Pop on encountering a closing bracket. Ensure the stack is empty at the end.
- Time Complexity: O(n)
- Space Complexity: O(n)

Pseudocode (C++)

```
bool isBalanced(string s) {
stack st;
for (char c  s) {
if (c == '(' || c == '{' || c == '[') st.push(c);
else {
if (st.empty()) return false;
if ((c == ')' && st.top()  =! '(') ||
(c == '}' && st.top()  =! '{') ||
(c == ']' && st.top()  =! '[')) return false;
st.pop();
        }
    }
```

return st.empty();
}

Dry Run
Input: "[{()}]"

Steps:
- Push [{, (.
- Pop (, {, [.

Output: true

5. Convert a Sentence into its Equivalent Mobile Numeric Keypad Sequence
- Approach 1: Map Characters to Numbers
- Trick: Map each character to its numeric equivalent based on a mobile keypad.
- Time Complexity: O(n)
- Space Complexity: O(1)

Pseudocode (C++)
string convertToKeypadSequence(string s) {
unordered_map keypad = {
{'a', "2"}, {'b', "22"}, {'c', "222"}, {'d', "3"}, {'e', "33"},
{'f', "333"}, {'g', "4"}, {'h', "44"}, {'i', "444"}, {'j', "5"},
{'k', "55"}, {'l', "555"}, {'m', "6"}, {'n', "66"}, {'o', "666"},

```
{'p', "7"}, {'q', "77"}, {'r', "777"}, {'s', "7777"}, {'t', "8"},
{'u', "88"}, {'v', "888"}, {'w', "9"}, {'x', "99"}, {'y', "999"},
{'z', "9999"}, {' ', "0"}
    };

string result = "";
for (char c  s) {
result += keypad[tolower(c)];
    }
return result;
}
```

Dry Run

Input: "hello world"

Conversion:

h: "44", e: "33", l: "555", o: "666", :0", w: "9", r: "777", d: "3"

Output: "4433555556660966677755553"

Medium Problems:

1. Check Whether One String is a Rotation of Another

Problem Statement:

Given two strings s1 and s2, check if s2 is a rotation of s1. A string rotation means you can rearrange s1 circularly to form s2.

Approach 1: Concatenate and

Search

- Idea: Concatenate s1 with itself and check if s2 exists as a substring in the concatenated string.
- Time Complexity: O(n)
- Space Complexity: O(n)

Pseudocode (C++)

```
bool isRotation(string s1, string s2) {
if (s1.length() != s2.length()) return false;
string temp = s1 + s1;
return temp.find(s2) != string::npos;
}
```

Dry Run:

Input: s1 = "abcd", s2 = "cdab"

Steps:

Concatenate: s1 + s1 = "abcdabcd"

Check substring: "cdab" found in "abcdabcd"

Output: true

2. Count and Say Problem

Problem Statement:

- The "Count and Say" sequence is defined as follows:
- Start with

"1".

- Count the number of consecutive characters in the previous term and construct the next term.
- Repeat until the nth term is generated.

Approach 1: Iterative Build

- Idea: Generate the sequence iteratively by counting consecutive characters in the previous sequence.
- Time Complexity: O(n * m) (where m is the average sequence length).
- Space Complexity: O(m).

Pseudocode (C++)

```
string countAndSay(int n) {
string result = "1";
for (int i = 1; i < n; i++) {
string temp = "";
int count = 1;
for (int j = 1; j < result.size(); j++) {
if (result[j] == result[j - 1]) count++;
else {
temp += to_string(count) + result[j - 1];
count = 1;
            }
        }
temp += to_string(count) + result.back();
```

```
result = temp;
    }
return result;
}
```

Dry Run:

Input: n = 4

Steps:

Term 1: "1"

Term 2: "11" (one 1)

Term 3: "21" (two 1s)

Term 4: "1211" (one 2, one 1)

Output: "1211"

3. Longest Common Prefix

Problem Statement:

Given a list of strings, find the longest common prefix among all strings.

Approach 1: Vertical Scanning

- Idea: Compare characters of each string column-wise until a mismatch occurs.
- Time Complexity: O(n * m) (where n is the number of strings, and m is the minimum string length).
- Space Complexity: O(1).

Pseudocode

(C++):

```
string longestCommonPrefix(vector& strs) {
if (strs.empty()) return "";
for (int i = 0; i < strs[0].size(); i++) {
char c = strs[0][i];
for (int j = 1; j < strs.size(); j++) {
if (i >= strs[j].size() || strs[j][i] != c)
return strs[0].substr(0, i);
        }
    }
return strs[0];
}
```

Dry Run:

Input: ["flower", "flow", "flight"]

Steps:

- Compare: f, l (common)
- Stop at mismatch on o and i.
- Output: "fl"

4. Print All Subsequences of a String

Problem Statement:

Generate and print all possible subsequences of a given

string.

Approach 1: Recursion

- Idea: Include or exclude each character in the string to form subsequences recursively.
- Time Complexity: O(2^n).
- Space Complexity: O(n).

Pseudocode (C++)

```
void generateSubsequences(string s, string current, int index) {
if (index == s.size()) {
cout << current << endl;
return;
    }
generateSubsequences(s, current + s[index], index + 1);
generateSubsequences(s, current, index + 1);
}
```

Dry Run:

Input: "abc"

Steps:

Subsequences: "abc", "ab", "ac", "a", "bc", "b", "c", "".

Output: All subsequences.

string="">

5. Find the Longest Palindrome in a String [Longest Palindromic Substring]

Problem Statement:

Given a string, find the longest substring that is a palindrome.

- Approach 1: Dynamic Programming
- Idea: Use a DP table to store if a substring is a palindrome and track the longest palindrome.
- Time Complexity: O(n^2).
- Space Complexity: O(n^2).

Pseudocode (C++):

```
string longestPalindrome(string s) {
int n = s.size(), start = 0, maxLen = 1;
vector> dp(n, vector(n, false));
for (int i = 0; i < n; i++) dp[i][i] = true;
for (int len = 2; len <= n; len++) {
for (int i = 0; i <= n - len; i++) {
int j = i + len - 1;
if (s[i] == s[j] && (len == 2 || dp[i + 1][j - 1])) {
dp[i][j] = true;
if (len > maxLen) {
start = i;
maxLen = len;
                }
```

```
}
        }
    }
return s.substr(start, maxLen);
}
```

Dry Run:

Input: "babad"

Steps:

Substrings: "bab", "aba".

6. Find the Longest Recurring Subsequence in a String

Problem Statement:

- Given a string, find the longest subsequence that appears at least twice in the string, with the same relative order but different positions.

- Approach 1: Dynamic Programming

- Idea: Use a DP table where dp[i][j] represents the length of the longest recurring subsequence for substrings s[0..i] and s[0..j]. Only consider characters at different indices.

- Time Complexity: O(n^2).

- Space Complexity: O(n^2).

Pseudocode

(C++):

```
int longestRecurringSubsequence(string s) {
int n = s.size();
vector> dp(n + 1, vector(n + 1, 0));
for (int i = 1; i <= n; i++) {
for (int j = 1; j <= n; j++) {
if (s[i - 1] == s[j - 1] && i != j)
dp[i][j] = dp[i - 1][j - 1] + 1;
else
dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
return dp[n][n];
}
```

Dry Run:

Input: "aab"

Steps:

DP Table:

Copy code

0 0 0 0

0 0 1 1

0 1 1 1

0 1 1 1

Result: "a" ẹhgth

1).

Output: 1

7. Word Wrap Problem

Problem Statement:

- Given a list of words and a maximum line width, arrange the words to minimize the total cost of wrapping lines. The cost of wrapping is calculated as the square of extra spaces in a line.

- Approach 1: Dynamic Programming

- Idea: Use a DP array where dp[i] represents the minimum cost to wrap words starting from word i. Calculate costs for every valid line break and take the minimum.

- Time Complexity: O(n^2).

- Space Complexity: O(n).

Pseudocode (C++)

```
int wordWrap(vector words, int maxWidth) {
int n = words.size();
vector dp(n + 1, INT_MAX);
dp[n] = 0;
for (int i = n - 1; i >= 0; i--) {
int width = 0;
for (int j = i; j < n; j++) {
width += words[j];
```

```
if (width > maxWidth) break;
int cost = (maxWidth - width) * (maxWidth - width);
dp[i] = min(dp[i], cost + dp[j + 1]);
width++;   //for the space
        }
    }
return dp[0];
}
```

Dry Run:

Input: words = [3, 2, 2], maxWidth = 6

Steps:

Possible splits:

Line 1: "3 2" (1extra space, cost 1).

Line 2: "2" (4extra spaces, cost 16).

Total: 1 + 16 = 17.

Output: 17

8. Rearrange Characters in a String such that No Two Adjacent are the Same

- Problem Statement:

- Rearrange characters in a string such that no two adjacent characters are the same. If not possible, return false.

- Approach 1: Priority

Queue

- Idea: Use a max-heap (priority queue) to always place the character with the highest frequency first.
- Time Complexity: O(n log n).
- Space Complexity: O(n).

Pseudocode (C++):

```
bool rearrangeString(string s) {
unordered_map freq;
for (char c   s) freq[c]++;
priority_queue> pq;
for (auto p   freq) pq.push({p.second, p.first});
string result = "";
pair prev = {-1, '#'};
while (!pq.empty()) {
auto curr = pq.top();
pq.pop();
result += curr.second;
if (prev.first > 0) pq.push(prev);
curr.first--;
prev = curr;
    }
return result.size() == s.size();
}
```

char="">

Dry Run:

Input: "aaabb"

Steps:

- Heap: [(3, 'a'), (2, 'b')].
- Arrange: "ababa".

Output: true

9. Minimum Characters to be Added at Front to Make String Palindrome

Problem Statement:

- Given a string, find the minimum characters to add at the front to make it a palindrome.

- Approach 1: KMP Algorithm
- Idea: Find the longest palindromic prefix using the KMP partial match table and add the remaining characters at the front.
- Time Complexity: O(n).
- Space Complexity: O(n).

Pseudocode (C++):

```
int minCharsToMakePalindrome(string s) {
string rev = s;
reverse(rev.begin(), rev.end());
string combined = s + "#" + rev;
```

```
vector lps(combined.size(), 0);
for (int i = 1; i < combined.size(); i++) {
int j = lps[i - 1];
while (j > 0 && combined[i] != combined[j]) j = lps[j - 1];
if (combined[i] == combined[j]) j++;
lps[i] = j;
    }
return s.size() - lps.back();
}
```

Dry Run:

Input: "abc"

Steps:

- Combined: "abc#cba".
- LPS: [0, 0, 0, 0, 0, 1, 2].
- Result: 3 - 2 = 1.
- Output: 1

10. Generate All Possible Valid IP Addresses from Given String

Problem Statement:

Given a string of digits, return all possible valid IP addresses that can be formed.

- Approach 1:

Backtracking

- Idea: Use backtracking to explore all possible partitions of the string into 4 valid parts. Check each part for validity.
- Time Complexity: O(3^4) (3 choices for each of 4 parts).
- Space Complexity: O(1).

Pseudocode (C++):

```
vector restoreIpAddresses(string s) {
vector result;
function backtrack = [&](string current, string remaining, int parts) {
if (parts == 4 && remaining.empty()) {
result.push_back(current);
return;
        }
if (parts >= 4 || remaining.empty()) return;
for (int len = 1; len <= 3; len++) {
if (len > remaining.size()) break;
string segment = remaining.substr(0, len);
if (stoi(segment) > 255 || (segment[0] == '0' && segment.size() > 1))
continue;
backtrack(current + (parts > 0  ?."  :"") + segment, remaining.substr(len),
parts + 1);
        }
    };
backtrack("", s, 0);
```